Universidad de los Andes > FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

Arquitectura De Computadores

Proyecto Nº 3

Grupo 2

Integrantes:

Edgardo Guianatti

Sebastián Letelier

Florencia Sáez

Cristóbal Toro

Profesores:

Jorge Gomez

4 de Noviembre de 2026

Resumen

Este informe documenta la implementación del Pochoco SoC (procesador Espino Core RV32E con el juego de reflejos del Proyecto 2) como un macro digital de celdas estándar, desde RTL hasta GDSII, utilizando LibreLane y el PDK GF180MCU. Se adaptó el RTL de FPGA a ASIC mediante un nuevo módulo superior asic_top, se extendió el assembler para generar una ROM combinacional del programa y se compararon dos arquitecturas de memoria

(A) ROM combinacional más RAM de datos de 64 palabras

(B) memoria unificada de 128 palabras implementada mediante flip-flops.

En la corrida original de la arquitectura A, con un período objetivo de 100 ns (10 MHz) y una utilización del 40 %, se obtuvo un WNS de setup de +28,1 ns en el corner lento y un área de die de 955.993 µm². Esta configuración presentó hold sin violaciones, cero errores LVS y cero violaciones de antena. La arquitectura B ocupa 1,86 veces el área de die de A, presenta un menor margen de setup (+18,9 ns) y conserva una violación de antena.

El barrido de período de la arquitectura A original mostró cierre temporal a 100 ns y 80 ns, e incumplimiento a 60 ns. Posteriormente, una optimización de la lógica de lectura de periféricos mejoró el WNS de setup de A hasta +39,4 ns, con un incremento del 0,28 % en el área de celdas. La implementación final seleccionada corresponde a la arquitectura A optimizada, que terminó sin violaciones de antena y con resultados positivos de setup y hold.

1. Introducción y objetivos

El Proyecto 3 parte del repositorio entregado en el Proyecto 2 (RTL modificado, `game.s` y assembler propio) y lo lleva a un macro implementado con celdas estándar. Los puertos del macro representan conexiones lógicas del juego (reloj, reset, botones, LEDs, displays de siete segmentos y SPI), no pads de I/O de un chip completo.

1.1 Objetivos específicos

- Crear asic_top.v sin elementos propios de FPGA e identificar y reemplazar las construcciones dependientes de esta plataforma.
- Implementar y comparar dos estrategias de memoria: ROM combinacional generada por el assembler más RAM de datos reducida (A), y memoria unificada implementada mediante flip-flops (B).
- Analizar los resultados de síntesis, identificar las celdas más utilizadas, caracterizar los recursos de almacenamiento de la RAM de datos y del register file, cuantificar el área del contador de ciclos y estudiar la ruta crítica.
- Proponer e implementar una optimización de RTL, evaluando su efecto en el área y la temporización.
- Realizar un barrido del período de reloj y de la utilización del core, estimar el costo asociado al área y verificar los resultados de signoff disponibles (DRC, LVS y antena).
- Comparar la simulación RTL con la simulación del netlist post-síntesis mediante el programa de prueba del juego.

1.2 Parámetros de la corrida final

Parámetro	**Valor**
CLOCK_PERIOD (T)	100 ns (10 MHz)
FP_CORE_UTIL (U)	40 %
Top-level	`asic_top`
Flujo	LibreLane 3.0.17 (Docker), flujo `classic`
PDK / biblioteca	GF180MCU `gf180mcuD`, rev. `54435919abffb937387ec956209f9cf5fd2dfbee`, `gf180mcu_fd_sc_mcu7t5v0`

2. Punto de partida

Resumen de lo que logramos en el Proyecto 2. El juego de reflejos enciende los cuatro LEDs al menos tres segundos, elige el LED objetivo con los dos bits bajos del contador de ciclos, mide el tiempo de respuesta en décimas de segundo, acumula diez respuestas correctas y muestra el promedio; ante un botón incorrecto muestra EE y reinicia la ronda conservando el progreso. El tiempo se mide con cycle_counter.v, un contador de 32 bits que se lee por MMIO en 0x8000000C. El assembler propio (tools/assembler.py, Python 3) traduce game.s a game.hex en dos pasadas. El detalle completo está en el informe del Proyecto 2, aquí decidimos conservar solo lo necesario para entender los cambios.

El único cambio de software exigido por el cambio de plataforma es la constante de tiempo: en la FPGA el reloj nominal era de 25 MHz y en el macro ASIC es de 10 MHz (T = 100 ns).

Constante en `game.s`	**Proyecto 2 (25 MHz)**	**Proyecto 3 (10 MHz)**	**Origen**
`THREE_SECONDS`	75.000.000	30.000.000	3 s × f
`DECISECOND`	2.500.000	1.000.000	f / 10
Período de 2^{32} ciclos	≈ 171,8 s	≈ 429,5 s	2^{32} / f

3. Del RTL de FPGA al RTL de ASIC

El módulo `asic/rtl/asic_top.v` instancia `pochoco_soc_asic` y expone solo puertos lógicos: `i_Clk`, `i_Reset_n`, `i_Switch[3:0]`, `o_LED[3:0]`, siete segmentos por cada uno de los dos displays (`o_Segment1_A..G`, `o_Segment2_A..G`) y los cuatro de SPI. Su parámetro `USE_UNIFIED_MEMORY` selecciona la arquitectura de memoria (0 = A, 1 = B). `pochoco_soc_asic.v` es una copia de `pochoco_soc.v` en la que se eliminaron el reset de encendido y la RAM de FPGA.

3.1 Construcciones dependientes de la FPGA

#	Construcción (RTL del P2)	Archivo	Por qué no sirve en ASIC	Reemplazo
1	`initial` con `$readmemh(MemFile, mem)`	`rtl/pochoco_ram.v`	En silicio no existe un archivo que se cargue al energizar; una memoria de FFs o celdas no tiene contenido definido al arrancar.	A: ROM combinacional `game_rom` generada por el assembler (`--rom`). B: el contenido se fija como valor de reset de los FFs (`generate_unified_memory.py`).
2	Memoria `reg [31:0] mem[0:511]` pensada para inferir BRAM	`rtl/pochoco_ram.v`	El PDK no ofrece BRAM inferible. Sin macro de memoria, 512 palabras se implementarían como 16.384 FFs más multiplexores.	Dimensionamiento según uso real: RAM de datos de 64 palabras (A) y memoria unificada de 128 palabras (B).
3	Primitiva iCE40 `SB_RAM40_4K` en `espino_regbank32`	`rtl/espino_core/espino_register_file.v`	Es una celda propia de la FPGA, inexistente en GF180MCU; el flujo no puede mapearla.	Register file con FFs y reset asíncrono (`asic/rtl/espino_register_file_asic.v`): 15 registros × 32 bits = 480 FFs (x0 es constante cero).
4	Reset de encendido `reg [3:0] por_cnt = 4'b0`	`rtl/pochoco_soc.v`	El valor inicial de un registro en la declaración es una propiedad de la FPGA (se carga con el bitstream). Un FF de silicio arranca en estado indefinido, por lo que el contador de reset no es confiable.	Puerto de reset externo `i_Reset_n` (activo en bajo, asíncrono) conectado directamente a `rst_ni`.

#	Construcción (RTL del P2)	Archivo	Por qué no sirve en ASIC	Reemplazo
5	Supuesto de reloj de 25 MHz y `i_Clk` directo	`game.s`, `pochoco_soc.v`	Las constantes de tiempo del programa dependen de la frecuencia; en el macro el reloj es de 10 MHz (T = 100 ns).	`THREE_SECONDS = 30.000.000`, `DECISECOND = 1.000.000`; reloj definido por `CLOCK_PORT/CLOCK_PERIOD`.
6	Botones y SPI leídos sin sincronizador	`rtl/pochoco_periph.v`, `pochoco_spi_slave.v`	Entradas asíncronas al reloj pueden causar metaestabilidad. El flujo no lo marca, pero sigue siendo un riesgo en silicio.	No se modificó.

Limitación de evidencia de síntesis: En el repositorio no se conservaron los registros originales completos de las corridas de síntesis. Por esta razón, no se atribuyen advertencias específicas ni números de línea a Yosys que no pudimos verificar. La identificación de las construcciones dependientes de FPGA se realizó mediante la inspección del RTL original del Proyecto 2 y su comparación con las modificaciones introducidas para la implementación ASIC del Proyecto 3.

Las construcciones identificadas y sus reemplazos se documentan en la tabla de esta sección. Esta comparación permite explicar las incompatibilidades arquitectónicas y las decisiones de adaptación realizadas, aunque no sustituye los registros originales de síntesis.

4. Memoria, programa del juego y extensión del assembler

4.1 Extensión del assembler

Se agregó la opción --rom a tools/assembler.py. Además de generar game.hex, la función generate_verilog_rom() escribe un módulo Verilog game_rom con un case sobre la dirección de byte, que devuelve la palabra de instrucción correspondiente. La dirección que no coincide con ninguna entrada devuelve una instrucción NOP (0x00000013). Esta implementación utiliza lógica combinacional sintetizable y no requiere $readmemh.

Para la arquitectura B, el script asic/scripts/generate_unified_memory.py lee game.hex y genera asic_unified_ram.v, donde las palabras del programa se establecen mediante los valores de reset de los flip-flops. Las posiciones no utilizadas se rellenan con instrucciones NOP.

Los comandos utilizados desde el directorio raíz del repositorio son:

```
python3 tools/assembler.py sw/game.s sw/game.hex --rom asic/rtl/memory/game_rom.v
python3 asic/scripts/generate_unified_memory.py
```

Un ejemplo de las entradas generadas en game_rom.v es:

```
32'd0:  data_o = 32'h800002b7;   // lui  x5, 0x80000
32'd4:  data_o = 32'h00028293;   // addi x5, x5, 0
32'd32: data_o = 32'h01c9c4b7;   // lui  x9, 0x01c9c  (parte alta de 30.000.000)
32'd36: data_o = 32'h38048493;   // addi x9, x9, 0x380 -> 0x01c9c380 = 30.000.000
```

4.2 Palabras usadas

Recurso	**Uso real**	**Cómo se obtuvo**
Instrucciones	87 palabras	`wc -l sw/game.hex` (87 líneas); el ensamblado de `game.s` regenera exactamente ese archivo. Coincide con las 87 entradas de `game_rom.v` y `mem[0..86]` de la memoria unificada.
Datos	0 palabras	Se decodificó `game.hex`: las 17 instrucciones de acceso a memoria (9 `lw` y 8 `sw`) usan como base el registro x5 = `0x80000000`, es decir, solo acceden a MMIO. El juego no usa pila ni variables en RAM; todo el estado vive en registros x5–x15.

Las 87 palabras de instrucciones y las 0 palabras de datos utilizadas se encuentran por debajo de los máximos considerados para el diseño: 256 palabras de instrucciones y 64 palabras de datos.

La RAM de datos de 64 palabras de la arquitectura A se mantiene a pesar de que el juego no realiza accesos a ella. Esto se debe a que el procesador conserva la posibilidad de direccionarla mediante instrucciones `lw` y `sw`. Por lo tanto, la memoria sigue siendo un recurso accesible por la lógica del procesador y no se elimina durante la síntesis.

4.3 Comparación de las dos estrategias

La comparación siguiente corresponde a las corridas originales de ambas arquitecturas, con un período de reloj de 100 ns y una utilización objetivo del 40 %.

Métrica (corrida original, 100 ns, U = 40 %)	**Arquitectura A**	**Arquitectura B**	**B / A**
Memoria	ROM comb. + RAM 64 pal.	Unificada 128 pal. (FFs)	—
Celdas sintetizadas	10.693	22.121	2,07
Flip-flops	2.851	4.901	1,72
Área de síntesis (µm²)	365.107,86	688.796,68	1,89
Área de die (µm²)	955.993	1.781.200	1,86
Utilización final de core	51,0 %	55,2 %	—
WNS setup, corner lento (ns)	+28,12	+18,86	—
Frecuencia máxima estimada* (MHz)	≈ 13,9	≈ 12,3	0,89
Violaciones de hold	0	0	—
Errores LVS	0	0	—
Violaciones de antena	0	1	—

Estimación como 1 / (T − WNS) con los datos de la corrida a 100 ns (A: 100 − 28,12 = 71,9 ns, B: 100 − 18,86 = 81,1 ns). Como cada período puede provocar una nueva optimización, es una cota orientativa y no un cierre demostrado.

Estos valores son estimaciones orientativas calculadas a partir de los resultados obtenidos a 100 ns. No representan un cierre temporal demostrado mediante corridas independientes a esos períodos, ya que cada cambio de CLOCK_PERIOD puede provocar nuevas optimizaciones de buffering, dimensionamiento y placement.

La arquitectura B implementa su memoria unificada de 128 palabras mediante 4.096 flip-flops (128 × 32). En cambio, la RAM de datos de A contiene 2.048 bits (64 × 32) implementados mediante flip-flops, mientras que su ROM de instrucciones utiliza lógica combinacional. Por lo tanto, al comparar específicamente estos bloques de almacenamiento secuencial, B utiliza 2.048 flip-flops más que la RAM de datos de A. Considerando el diseño completo, la síntesis contabiliza 2.050 flip-flops adicionales en B respecto de A (4.901 frente a 2.851).

Además, la memoria unificada de B deja 41 palabras sin utilizar, ya que el programa ocupa 87 de sus 128 posiciones (128 − 87). Una ventaja de esta arquitectura es que permite modificar el contenido del programa durante la ejecución, característica que no resulta necesaria para el juego implementado.

En términos generales, la arquitectura A presenta resultados más favorables en las métricas de área, cantidad de flip-flops, margen de setup y violaciones de antena. Ambas arquitecturas presentan cero violaciones de hold y cero errores LVS. La diferencia de área de die muestra que B ocupa aproximadamente 1,86 veces el área de A, mientras que su área de síntesis es 1,89 veces mayor.

Por estas razones, seleccionamos la arquitectura A para la implementación final, ya que satisface las necesidades del juego y permite obtener un diseño de menor área y con mayor margen de temporización.

5. Síntesis y trazabilidad de recursos

5.1 Cinco celdas más usadas (arquitectura A)

#	Celda	Cantidad	% celdas	Parte del diseño que lo explica (inferido del RTL)
1	mux2_2	2.568	24,02	Habilitación de escritura: los FFs sin enable (dffq) necesitan un mux por bit: 2.048 (RAM de datos) + 480 (register file) = 2.528, más ≈ 40 en otras partes.
2	dffq_1	2.110	19,73	FFs sin reset: los 2.048 bits de la RAM de datos más ≈ 62 de registros de salida (ROM, RAM).
3	nand2_1	1.360	12,72	Árboles de lectura de la RAM (64:1 × 32 bits), puertos de lectura del register file, decodificadores y lógica de la ALU.
4	aoi22_1	796	7,44	Multiplexado de lectura (selección AND-OR) de la RAM y de los dos puertos del register file.
5	dffrnq_1	738	6,90	FFs con reset asíncrono: register file (480), contador de ciclos (32), registros del núcleo, de periféricos y de SPI.

5.2 Aporte de área por módulo

El contador de ciclos agregado en el Proyecto 2 ocupa 3.714,28 µm², lo que equivale al 1,02 % del área de síntesis de la arquitectura A (365.107,86 µm²). Su implementación considera 32 flip-flops con reset y un incrementador de 32 bits.

Por conteo de elementos de almacenamiento, la RAM de datos y el register file son los principales recursos de este tipo. La RAM contiene 64 palabras de 32 bits (2.048 bits), mientras que el register file utiliza 480 flip-flops. Sin embargo, no se dispone de un desglose completo y verificable del área sintetizada por módulo para la arquitectura A. Aunque existe la configuración asic/config/config_A_hierarchical.json, su presencia no permite afirmar que se haya obtenido dicho desglose. Por ello, el siguiente análisis se basa en los elementos identificados en el RTL y en los resultados globales de síntesis.

Módulo o recurso	Elementos identificados	Interpretación
RAM de datos	2.048 bits	Principal consumidor de almacenamiento
Register file	480 FFs	Almacenamiento de los registros del procesador
Contador de ciclos	32 FFs	Contador de 32 bits agregado en el Proyecto 2
ALU	Lógica combinacional	Operaciones aritméticas y lógicas
ROM de instrucciones	87 palabras	Implementación mediante lógica combinacional
Periféricos	Registros y lógica de control	Lectura de botones, LEDs, displays y SPI

La RAM de datos representa un consumo importante de recursos porque sus bits se implementan mediante flip-flops y lógica adicional de lectura y escritura, en lugar de utilizar una macro de memoria específica del PDK. El register file y el contador de ciclos también contribuyen al almacenamiento del diseño.

En conjunto, la RAM de datos, el register file y el contador representan 2.560 elementos de almacenamiento, aproximadamente el 89,8 % de los 2.851 flip-flops contabilizados en la arquitectura A. Este porcentaje permite dimensionar su participación en el almacenamiento total, pero no debe interpretarse como su porcentaje del área física del circuito.

5.3 Comparación con lo esperado

Antes de la síntesis, podía esperarse que el núcleo del procesador, compuesto por la ALU, la unidad de control, el decodificador y el register file, representara una parte importante del área total.

Sin embargo, el análisis de los recursos de almacenamiento muestra que la RAM de datos tiene un impacto considerablemente mayor al que podría suponerse inicialmente. Sus 2.048 bits, implementados mediante flip-flops y lógica de selección, superan ampliamente los 480 flip-flops del register file.

Esto se explica por la ausencia de una macro de memoria SRAM utilizada por esta implementación. En consecuencia, el almacenamiento de la RAM se construye con celdas

estándar, aumentando tanto el número de flip-flops como la lógica combinacional necesaria para acceder a ellos.

El resultado evidencia que, para este diseño, la arquitectura de memoria tiene un impacto determinante sobre el área, incluso cuando el programa del juego no utiliza posiciones de la RAM de datos.

5.4 Trazabilidad

La síntesis aplana la jerarquía, pero conserva los nombres de los registros en los nombres de señal de los FFs. Se usó ese método (`asic/scripts/locate_registers.py`): se buscan en el netlist final (`final/nl/asic_top.nl.v`) los FFs cuya salida `Q` contiene `u_rf.registers[` (register file) o `u_periph.cycles[` (contador de ciclos) y se obtienen sus coordenadas del DEF. Resultado: 480 FFs del register file y 32 del contador (CSV en `reports/layout/`). La ALU es lógica combinacional sin registros propios, por lo que este método no puede identificarla.

6. Timing

6.1 Barrido de período (arquitectura A)

Se ejecutó la arquitectura A con tres períodos, incluyendo T = 100 ns. Una corrida cierra cuando WNS ≥ 0 y TNS = 0 para setup y hold en todos los corners.

T (ns)	f (MHz)	Setup WNS typ (ns)	Setup WNS lento (ns)	Setup TNS lento (ns)	Hold WNS typ (ns)	Hold WNS lento (ns)	Cierra
100	10,0	+62,19	+28,12	0	+0,387	+0,970	Sí
80	12,5	+42,19	+8,12	0	+0,387	+0,970	Sí
60	16,7	+22,19	−11,88	−4.373,1	+0,387	+0,970	No

El TNS de setup en el corner typical es 0 en los tres casos, y el TNS de hold es 0 en todos los corners. El peor margen de hold entre los corners evaluados (worst_hold_wns) es +0,137 ns. El menor período evaluado que cumple los criterios de cierre es 80 ns (12,5 MHz). Aunque el WNS de setup a 100 ns permite estimar un límite cercano a 72 ns, no se ejecutaron corridas entre 60 y 80 ns, por lo que no se afirma un período mínimo de cierre más preciso. Además, cada valor de CLOCK_PERIOD puede provocar nuevas optimizaciones del circuito, como buffering, dimensionamiento y placement, de modo que los resultados de distintos períodos no son directamente extrapolables.

6.2 Ruta crítica

Elemento	Valor (`baseline_A_100ns.rpt`, corner `max_ss_125C_4v50`)
Celda de inicio	_15824_ (dffq_1), salida u_pochoco.gen_split_memory.u_instr_rom.rdata_o[12], correspondiente a un registro de salida de la ROM de instrucciones.
Celda de fin	_18450_ (dffrnq_1, con reset). Según nuestro análisis del RTL, correspondería al registro de lectura de periféricos (rdata_o de pochoco_periph); esta asociación no se verificó directamente en el netlist.
Tiempos	Llegada de datos: 74,89 ns; requerido: 103,01 ns; slack: +28,12 ns. El reporte considera una incertidumbre de reloj de 0,25 ns y un tiempo de setup de la celda de 0,57 ns.
Estructura	Aproximadamente 14 compuertas lógicas (nand, nor, oai, aoi y and), además de buffers dlyb_1 presentes en la ruta.

Interpretación microarquitectónica. A partir de las señales de inicio y término reportadas y de la inspección del RTL, interpretamos que esta ruta corresponde al procesamiento de una instrucción desde la etapa de fetch hasta la captura de un dato de lectura de periféricos. La ruta atraviesa lógica de decodificación, selección de dirección y lectura de datos, antes de que el resultado pueda utilizarse en la etapa posterior correspondiente.

Esta interpretación permite relacionar la ruta crítica con la lógica de control y lectura de periféricos, en lugar de atribuirla directamente a una operación aritmética de la ALU. Sin embargo, la instancia de término identificada en el reporte es _18450_ (dffrnq_1) y no se dispone de una correspondencia jerárquica confirmada en el netlist final. Por ello, la asociación específica con el registro rdata_o se presenta como una interpretación basada en el análisis del RTL y no como una identificación verificada de la instancia física.

6.3 Optimización propuesta e implementada

Se evaluaron dos cambios de RTL:

- **Contador de ciclos segmentado (cycle_counter_optimized.v, acarreo por bytes)**: Se verificó su equivalencia con el original durante 100.000 ciclos, pero aumentó el área del contador de 3.714,28 a 3.813,06 µm² (+2,66 %), por lo que se descartó y se mantuvo el contador original.
- **Lógica de lectura de periféricos (pochoco_periph.v)**: se reemplazó la selección con case por una combinación con máscaras de bits: read_value = ({32{read_buttons}} & btn) | ({32{read_cycles}} & cycles). La equivalencia lógica se comprobó con Yosys (asic/verification/check_periph_equiv.sh).

Resultados físicos de A (corrida original frente a la optimizada final):

Métrica	**Original**	**Optimizada**	**Diferencia**
Área de celdas (µm²)	464.441	465.747	+1.306 (+0,28 %)
Área de core (µm²)	910.038	914.854	+4.816 (+0,53 %)
Área de die (µm²)	955.993	958.919	+2.926 (+0,31 %)
Setup WNS lento (ns)	28,119	39,410	+11,29 (+40,2 %)
Setup TNS (ns)	0	0	—
Hold WNS (ns)	0,1367	0,1154	−0,0213
Errores route DRC / Magic DRC / LVS	0 / 0 / 0	0 / 0 / 0	—

Métrica	Original	Optimizada	Diferencia
Violaciones de antena	0	0	—

El cambio mejoró el slack de setup en aproximadamente 11,3 ns, con un incremento del 0,28 % en el área de celdas reportada. La mejora es coherente con la simplificación de la lógica de selección de lectura de periféricos, aunque los resultados por sí solos no permiten atribuir todo el cambio exclusivamente a la eliminación de compuertas o a una reducción específica del fanout.

El hold WNS disminuyó levemente, desde +0,1367 ns hasta +0,1154 ns, pero se mantuvo positivo y con TNS igual a cero. Una primera corrida que incorporó solamente el cambio de RTL entregó un WNS de +39,49 ns. La corrida final, utilizando la configuración orientada a la reparación de antena, obtuvo +39,41 ns (reports/optimization/). El barrido de período no se repitió sobre el diseño optimizado, por lo que su frecuencia máxima no está medida mediante corridas específicas.

6.4 Constantes de game.s para T = 100 ns

Con T = 100 ns, f = 1 / T = 10 MHz. Para mantener la espera de tres segundos y la unidad de décimas de segundo:

```
THREE_SECONDS = 3 s × 10.000.000 ciclos/s = 30.000.000 ciclos   (0x01C9C380)
DECISECOND    = 10.000.000 / 10         =  1.000.000 ciclos   (0x000F4240)
T_i = floor(ΔC / 1.000.000)   (décimas de segundo)
```

Las constantes se cambiaron en `.equ THREE_SECONDS, 30000000` y `.equ DECISECOND, 1000000`, y se regeneró el programa con el assembler: la palabra de `li x9, THREE_SECONDS` es `lui 0x01c9c` + `addi 0x380`, que suma 0x01C9C380 = 30.000.000 (visible en `game_rom.v`). El contador de 32 bits ahora tarda 2^{32} / 10 MHz ≈ 429,5 s en desbordar, lo que sigue siendo transparente para las restas del programa.

7. Floorplan, placement y costo

7.1 Exploración de utilización

Se implementó la arquitectura A (original, T = 100 ns) con FP_CORE_UTIL de 30, 40 y 50 %.

U objetivo	Área de die (µm²)	Setup WNS lento (ns)	Violaciones de antena	Magic DRC / LVS
30 %	1.266.870	+31,56	1	0 / 0
40 %	955.993	+28,12	0	0 / 0
50 %	768.917	≈ +36,0 (leído del gráfico)	0	0 / 0

Las tres corridas completaron el flujo sin errores de Magic DRC ni de LVS. La utilización objetivo del 50 % produjo el menor área de die entre los puntos evaluados (768.917 µm²), sin violaciones de antena y con un WNS de setup positivo. El caso de 30 % presentó una violación de antena, mientras que los casos de 40 % y 50 % no presentaron violaciones de este tipo. No se probó una utilización superior al 50 %, por lo que este valor representa el mejor resultado entre los puntos evaluados y no necesariamente el límite del flujo.

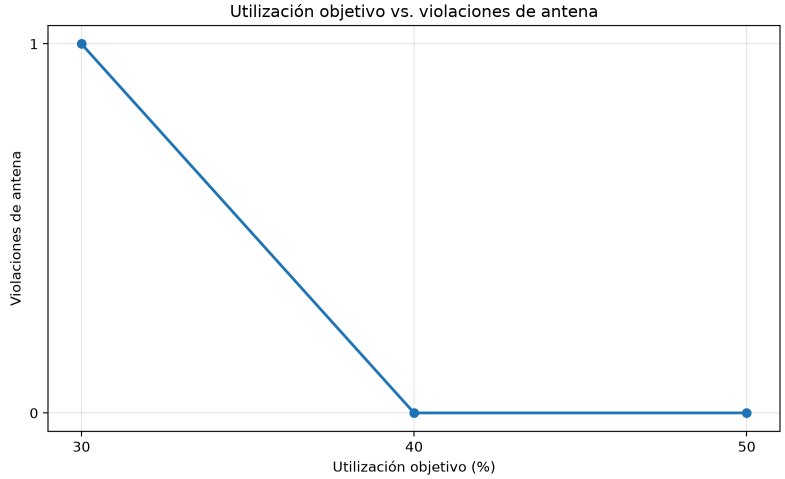


Figura 1. Utilización objetivo frente a violaciones de antena (fuente: reports/utilization/).

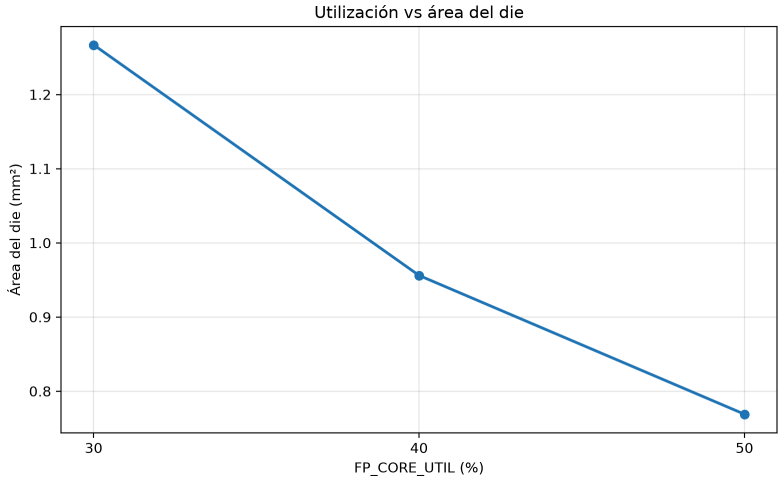


Figura 2. Utilización objetivo frente a área de die.

7.2 Terminales del macro

Los 30 terminales (28 señales más VDD/VSS) están registrados en reports/layout/pin_coordinates.csv. La mayoría de las señales queda ubicada en el borde derecho del macro (x = 970,045 µm). Las excepciones son i_Clk, que se encuentra en la esquina inferior (y = 27,2 µm), e i_Reset_n, ubicado en el borde superior (x = 892,9 µm).

El resto de los terminales se agrupa por función en la zona y ≈ 609–677 µm: botones y LEDs (y ≈ 609–632), display 1 (y ≈ 634–641), y SPI junto con display 2 (y ≈ 663–677). Esta distribución presenta una agrupación funcional de las señales y permite mantener cercanos los terminales asociados a cada conjunto de periféricos.

Sin embargo, los archivos de configuración utilizados no contienen restricciones explícitas de ubicación de pines (FP_PIN_ORDER_CFG). Por lo tanto, esta distribución corresponde a la asignación realizada automáticamente por la herramienta durante el flujo físico y no a una ubicación definida manualmente por nosotros.

Terminales frente a pads de I/O: Los terminales son puntos de conexión lógica del macro en capas de metal y no incorporan por sí mismos circuitos de protección. Un chip completo necesitaría pads de I/O con protección ESD, buffers de salida capaces de manejar la carga externa (LEDs y displays) y, en las entradas, protección y acondicionamiento de señales, por ejemplo, sincronizadores para los botones. Además, un anillo de alimentación y el empaquetado del chip quedan fuera del alcance de este macro.

7.3 Costo estimado

Con US$ 23.000 por mm² y suponiendo costo lineal con el área de die:

Diseño	Área de die (mm²)	Costo estimado (US$)
A, U = 50 %	0,7689	17.685
A, U = 40 % (original)	0,9560	21.988
A optimizada final, U = 40 %	0,9589	22.055
A, U = 30 %	1,2669	29.138
B (memoria unificada), U = 40 %	1,7812	40.968

Limitaciones. El costo real no es lineal con el área, depende del número de dies por oblea y del rendimiento (yield), de costos fijos de máscara y de empaquetado, y de que un macro necesita un chip completo con anillo de pads y marco de sellado (seal ring) alrededor. Además, el área del die incluye espacio vacío fijado por la utilización objetivo, no solo celdas.

Decisión de mayor impacto. La elección de la arquitectura de memoria tiene el mayor impacto en el costo estimado. La arquitectura B presenta un costo aproximado de US$ 18.980 más que la arquitectura A original con U = 40 %, lo que representa un incremento cercano al 86 %. En términos de almacenamiento, B utiliza 4.096 FF para su memoria unificada de 128 palabras, mientras que la RAM de datos de A utiliza 2.048 FF. Por lo tanto, B requiere aproximadamente 2.048 FF adicionales para implementar estos bloques de memoria. La diferencia total observada en la síntesis es de 2.050 FF al considerar también los demás registros del diseño.

En comparación, variar la utilización entre 30 % y 50 % produce una diferencia de costo estimado de aproximadamente US$ 11.454. La optimización de RTL de la arquitectura A a U = 40 % aumenta el costo estimado en aproximadamente US$ 67 respecto de la corrida original, debido al pequeño incremento del área de die. Por estas razones, la reducción de la memoria de datos constituye una oportunidad relevante para disminuir el área y el costo estimado de la implementación.

8. Signoff y layout

8.1 Resultados finales (arquitectura A optimizada, corrida architecture_A_optimized_antenna)

Verificación	Resultado	Comentario
Route DRC	0 errores	Reporte de ruteo detallado
Magic DRC	0 errores	Limpio
LVS	0 errores	Limpio
Violaciones de antena	0	Con reparación configurada (`GRT/DRT_ANTENNA_REPAIR_ITERS` = 5, margen 30)
Setup (WNS lento / TNS)	+39,41 ns / 0	Cumple
Hold WNS	+0,115 ns, TNS = 0	Cumple
KLayout DRC independiente	No ejecutado	Solo se usó Magic DRC

Las corridas originales de las arquitecturas A y B también reportan indicadores asociados a slew y capacitancia. Los valores disponibles son 4.576 y 33 para A, y 21.558 y 1.435 para B, respectivamente. Se conservan como valores reportados por las corridas originales, sin interpretarlos como cantidades de violaciones ni extrapolarlos a la implementación optimizada.

8.2 Violaciones de antena

Arquitectura B. final_antenna.rpt reporta una violación: la red net805, en el pin fanout804/I (dlyb_1), en la capa Metal2, con razón de área lateral parcial de 467,62 frente al máximo de 400,00 (razón P/R = 1,17). La violación corresponde a una conexión que supera el límite de antena establecido para esa capa. El ciclo de reparación (after_repair.rpt) quedó vacío, pero el reporte final aún muestra la infracción. Esto representa un riesgo de daño al óxido de compuerta durante la fabricación. Como B no es el macro final seleccionado, lo documentamos como una limitación de esa arquitectura.

Arquitectura A. La corrida con U = 30 % tuvo una violación de antena (utilization_comparison.csv), las corridas con U = 40 % y U = 50 % no presentaron violaciones de este tipo. La corrida final se ejecutó con la configuración config_A_optimized_antenna.json, que incorpora ajustes orientados a la reparación de violaciones de antena, y terminó con 0

violaciones. Sin embargo, los antecedentes disponibles no incluyen el detalle de la violación de la corrida U = 30 % (red, pin, capa y ubicación en el layout), ni permiten confirmar qué mecanismo específico de reparación se aplicó. Por lo tanto, no es posible atribuirle una inserción concreta de diodos o un cambio de capa. El resultado final sí permite concluir que la configuración seleccionada terminó sin violaciones de antena reportadas.

8.3 Layout anotado y trazabilidad

La figura 3 muestra la ubicación física de los 480 FFs del register file (azul) y los 32 FFs del contador de ciclos (rojo), obtenida del DEF final con `locate_registers.py`. El register file ocupa la parte superior central del core (x ≈ 255–640 µm, y ≈ 580–975 µm) y el contador queda disperso en la zona inferior derecha (x ≈ 535–855 µm, y ≈ 505–585 µm).

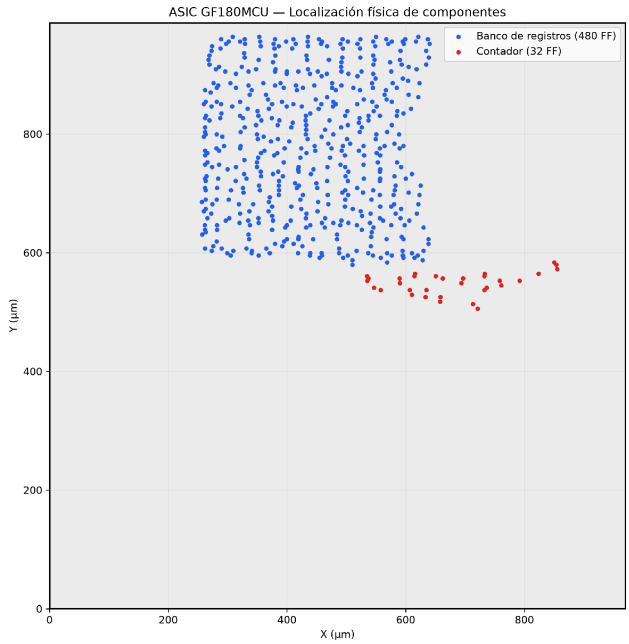


Figura 3. Ubicación de los FFs del register file y del contador de ciclos (graficadas con matplotlib).

Limitaciones declaradas.

1- La Figura 3 corresponde a una visualización generada a partir de las coordenadas del DEF y no a una captura anotada directamente desde KLayout u OpenROAD GUI. Por ello, permite observar la distribución de los registros identificados, pero no reemplaza una captura del layout físico completo.

2- No fue posible identificar con certeza la ubicación física de la ALU, debido a que Yosys aplanó la jerarquía durante la síntesis y no se conservaron nombres de instancias que permitieran distinguir sus celdas en el layout final. En consecuencia, no se presenta una ubicación de la ALU como si estuviera verificada.

Los archivos de layout entregados corresponden a gds/architecture_A.gds (arquitectura A optimizada final) y gds/architecture_B.gds (arquitectura B original).

9. Simulación funcional post-síntesis

Se simuló el diseño asic_top con Icarus Verilog en dos formas: RTL y netlist post-síntesis, sin retardos anotados. Para ambas simulaciones se utilizó el mismo testbench (asic/verification/tb_compare.v), con un reloj de 100 ns y una secuencia de pulsaciones aplicada automáticamente en ciclos definidos. Para acortar la simulación se utilizó el programa game_test, de 85 instrucciones, con las constantes THREE_SECONDS = 100 y DECISECOND = 20 ciclos, en lugar de los valores de la implementación final (30.000.000 y 1.000.000). Ambas simulaciones utilizaron la misma configuración de prueba (game_rom_test.v); el netlist fue generado mediante config_A_test.json.

Evento	**Ciclo RTL**	**Ciclo gate-level**	**Resultado**
Arranque (LEDs = 1111)	20	20	Coincide
Selección de LED objetivo (`0010`)	146	146	Coincide
Pulsación correcta	146	146	Coincide
Avance a la siguiente ronda (LEDs = 1111)	210	210	Coincide
Segunda selección (`0001`)	335	335	Coincide
Pulsación incorrecta	335	335	Coincide
Reinicio por error (LEDs = 0000)	365	365	Coincide

Los siete eventos coinciden exactamente, y el testbench comprueba que tras una pulsación incorrecta los LEDs se apagan (reinicio de la ronda actual). Evidencia: `verification_results.txt`, `rtl_results.log` y `gate_results.log`.

Análisis de resultados. Los siete eventos coinciden exactamente en los ciclos registrados para ambas simulaciones. Esta coincidencia indica que, para la secuencia de estímulos aplicada, la síntesis preservó el comportamiento funcional observado en las salidas de los LEDs. En particular, se verificaron el arranque, la selección de objetivos, el avance de ronda y el reinicio después de una pulsación incorrecta. Estos resultados respaldan la equivalencia funcional de los eventos evaluados entre RTL y netlist post-síntesis, aunque no constituyen una demostración formal de equivalencia para todos los estados y entradas posibles.

Alcance y limitaciones. La prueba compara ciclos de eventos observables en los LEDs, no lee el valor del tiempo mostrado en los displays ni el ΔC medido, por lo que la igualdad del número de ciclos medidos se infiere de la igualdad de ciclos de los eventos. Tampoco cubre todas las instrucciones ni situaciones, y no incluye retardos SDF. Por tanto, estos resultados validan el comportamiento funcional de la secuencia probada, pero no permiten concluir que el circuito mantenga los mismos tiempos de propagación físicos después del layout.

10. Reproducibilidad

10.1. Entorno y versiones

El flujo de implementación ASIC se ejecutó utilizando LibreLane 3.0.17 mediante Docker, con el PDK gf180mcuD, revisión 54435919abffb937387ec956209f9cf5fd2dfbee, y la biblioteca estándar gf180mcu_fd_sc_mcu7t5v0.

Las herramientas auxiliares utilizadas incluyen Icarus Verilog para simulación y Python 3 para la generación de memorias y ejecución de scripts. No se dispone del registro de versiones internas de Yosys, OpenROAD y Magic correspondientes a la imagen Docker utilizada, por lo que no se especifican números de versión para estas herramientas.

10.2. Generación del programa y memorias

Desde el directorio raíz del repositorio, ejecutar:

python3 tools/assembler.py sw/game.s sw/game.hex --rom asic/rtl/memory/game_rom.v

python3 asic/scripts/generate_unified_memory.py

Estos comandos permiten generar el archivo hexadecimal del programa y actualizar los archivos de memoria utilizados por el diseño.

10.3. Ejecución del flujo ASIC

La corrida final corresponde a la arquitectura A optimizada, utilizando la configuración asic/config/config_A_optimized_antenna.json.

El comando utilizado para ejecutar el flujo es:

librelane --dockerized --pdk gf180mcuD --flow classic --run-tag architecture_A_optimized_antenna asic/config/config_A_optimized_antenna.json

La ejecución requiere un entorno en el que el comando librelane esté disponible y Docker se encuentre configurado. Por lo tanto, este comando debe ejecutarse desde el mismo tipo de entorno utilizado para las corridas originales.

Las configuraciones para la arquitectura B y los experimentos de período de reloj (60 y 80 ns) y utilización (30 y 50 %) se encuentran en asic/config/. Para reproducirlos se utiliza el mismo procedimiento, cambiando el archivo JSON y la etiqueta --run-tag según el experimento.

Los directorios de resultados generados por LibreLane contienen los reportes y archivos intermedios de cada corrida. Estos resultados no están incluidos completamente en el repositorio disponible, por lo que su reproducción requiere ejecutar nuevamente las configuraciones correspondientes.

10.4. Verificación funcional

La verificación funcional compara los ciclos en que ocurren siete eventos observables en las salidas LED entre la simulación RTL y la simulación del netlist post-síntesis.

La prueba utiliza el testbench asic/verification/tb_compare.v y registra los resultados en los siguientes archivos:

- verification_results.txt
- rtl_results.log
- gate_results.log

Los siete eventos evaluados presentan coincidencia exacta entre ambas simulaciones. El testbench también comprueba que, después de una pulsación incorrecta, los LEDs se apagan como parte del reinicio de la ronda actual.

Alcance de la verificación: la prueba compara los ciclos de eventos observables en los LEDs. No comprueba directamente el valor del tiempo mostrado en los displays ni el valor de ΔC medido. Por lo tanto, la igualdad de los ciclos de los eventos no debe interpretarse como una comparación directa de los valores mostrados en los displays. Asimismo, la prueba no cubre todas las instrucciones ni todas las situaciones posibles del diseño y no incorpora retardos SDF.

11. Conclusiones

El Pochoco SoC con el juego de reflejos fue implementado y llevado a un macro ASIC sobre la tecnología GF180MCU. La arquitectura A, basada en una ROM combinacional y una RAM de datos reducida, logró cumplir los criterios de cierre a un período de reloj de 100 ns y una utilización del 40 %, con resultados positivos de setup y hold, LVS sin errores y sin violaciones de antena en su configuración final optimizada.

La comparación entre arquitecturas permitió identificar que la organización de memoria tiene un impacto significativo en el área del diseño. La arquitectura A obtuvo un área de die aproximadamente un 46 % menor que la arquitectura B, que utiliza una memoria unificada implementada mediante flip-flops. Esto evidencia el costo de implementar estructuras de memoria mediante lógica secuencial estándar, especialmente cuando su capacidad supera las necesidades reales de la aplicación. En particular, la RAM de datos de la arquitectura A no recibe accesos durante la ejecución del juego, por lo que reducir su capacidad constituye una oportunidad concreta de optimización.

En cuanto a temporización, el análisis identificó una ruta crítica asociada al procesamiento de instrucciones y a la selección de datos de lectura de periféricos. La modificación de esta lógica mediante una selección basada en máscaras AND/OR permitió mejorar el slack de setup desde +28,119 ns hasta +39,410 ns, con un incremento de área de celdas de síntesis de solo un 0,28 %. Por otra parte, el barrido de períodos mostró que el diseño original cumple el cierre temporal a 100 ns y 80 ns, mientras que presenta incumplimiento a 60 ns, delimitando así el rango de operación evaluado.

En conjunto, los resultados muestran que la arquitectura de memoria y la lógica de selección de periféricos son factores relevantes para optimizar el área y la temporización del Pochoco SoC. El trabajo permitió obtener una implementación física funcionalmente verificada a nivel de ciclos de eventos y caracterizar sus principales compromisos de diseño, dejando como oportunidades de mejora la reducción de la memoria de datos y el análisis de temporización a períodos inferiores.